# How the AI Receptionist answers a chat message

A step-by-step walkthrough of the conversation engine, with diagrams, plus a learning path for the concepts behind it.

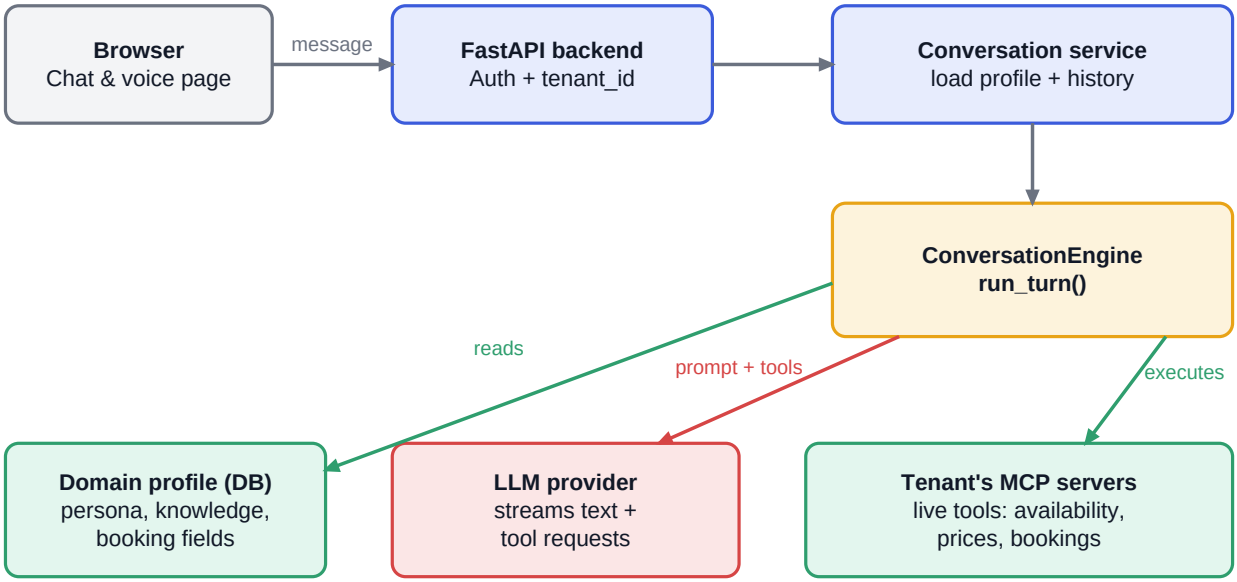
| Based on these files | |
|---|---|
| conversation/engine.py | Runs one turn: prompt, LLM, tool loop, streamed text |
| conversation/prompt.py | Builds the system prompt from the domain profile |
| domain/catalog.py | The business domains and each new business's starting profile |
| mcp/tools.py | Loads and runs a tenant's MCP tools |

## Contents


- 1. The big picture
- 2. One chat turn, step by step
- 3. The tool loop (sequence diagram)
- 4. Anatomy of the system prompt
- 5. Why a new admin gets "I don't have that information"
- 6. Concepts to learn, in order
- 7. Glossary and practice exercises

# 1. The big picture

The assistant is a **domain-agnostic engine**. The same code serves a hotel, a gym or a hospital. What changes per business is data: its **domain profile** (persona, knowledge base, booking fields) and its **MCP tools** (live access to that business's own system).



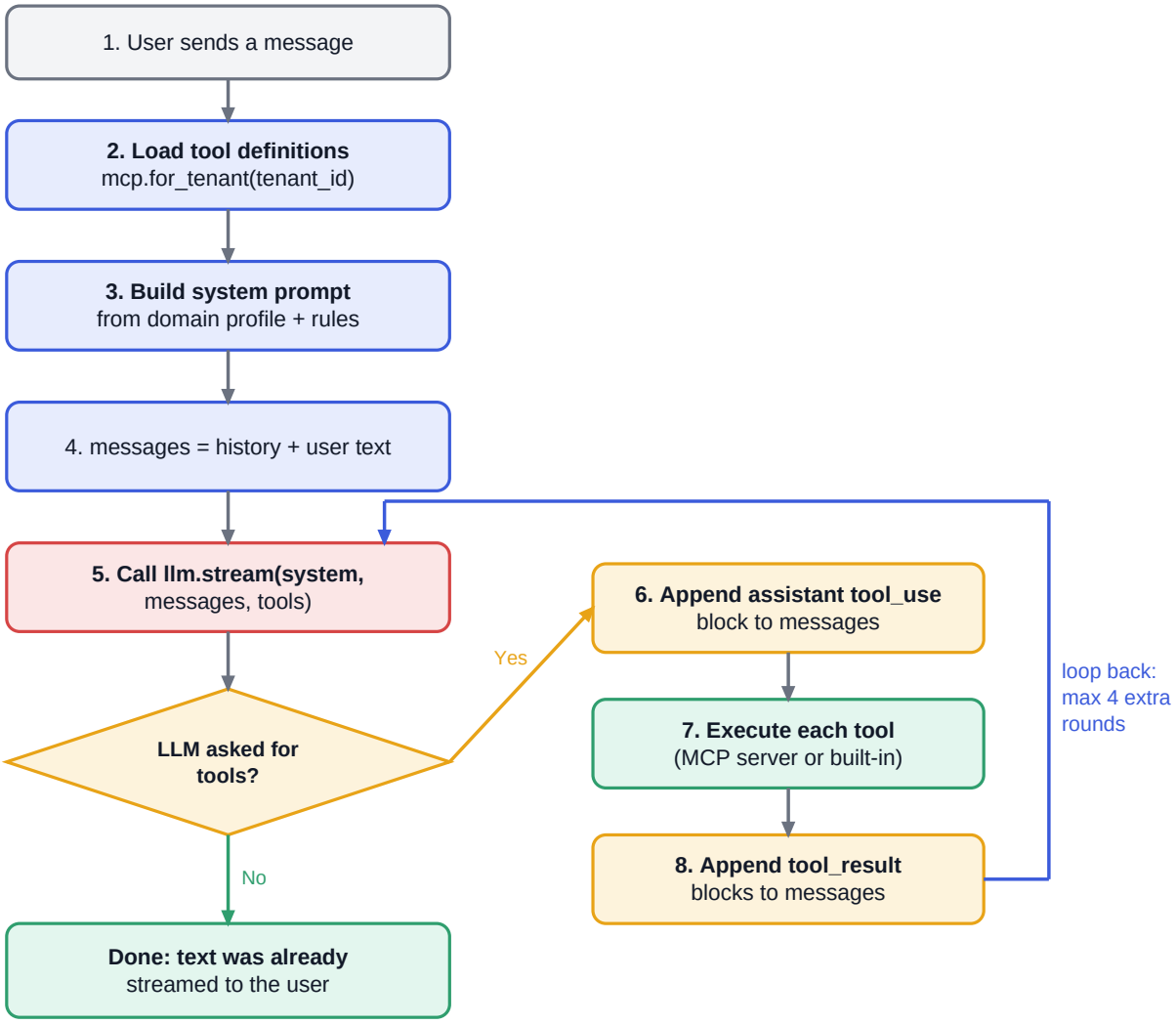

Everything is scoped by tenant_id taken from the logged-in session, never from client input.

## Three sources of truth feed every answer:

| Source | What it gives | When it is empty |
|---|---|---|
| **Domain profile** | Who the assistant is (persona, greeting, tone) and the fields a booking needs. | Never empty: created with the business from its domain. |
| **Knowledge base** | Static facts: FAQs and info such as hours, policies, address. | Empty for a new business. The assistant knows no facts. |
| **MCP tools** | Live data and actions: availability, prices, create or cancel booking. | Empty until the admin connects an MCP server. |

# 2. One chat turn, step by step

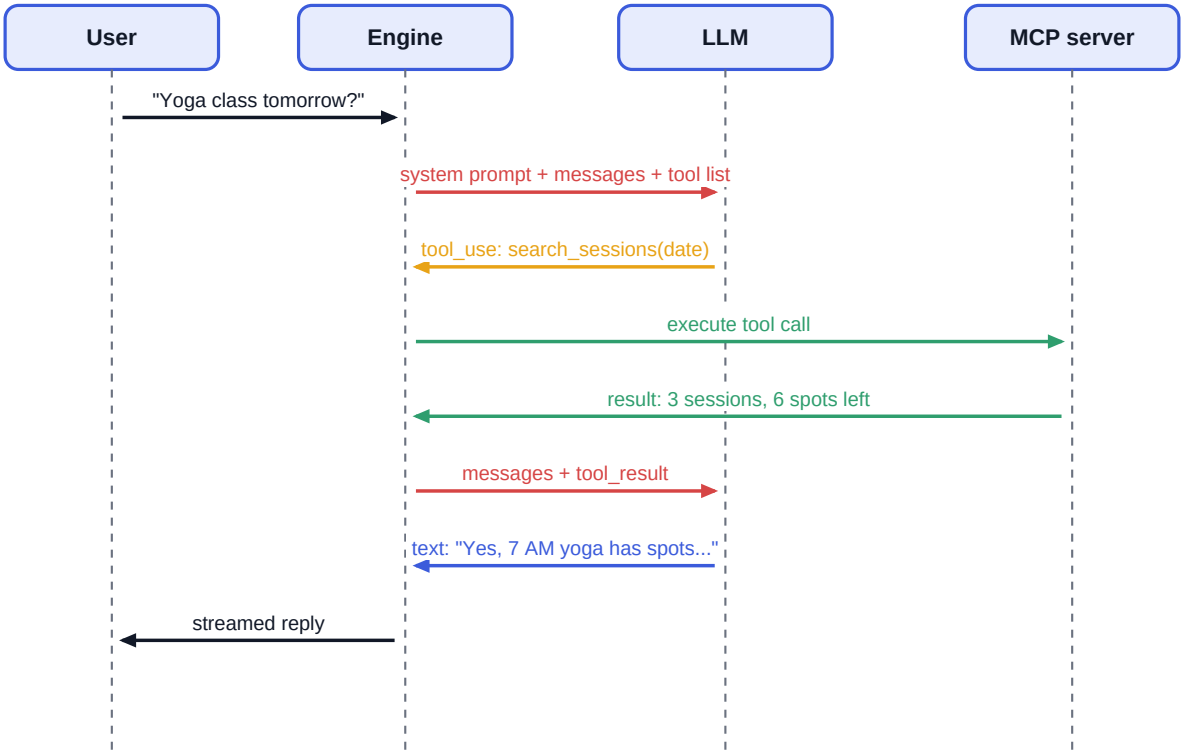
Everything below happens inside `ConversationEngine.run_turn()` for each message the user sends. The engine is pure with respect to storage: the caller passes history in and saves results.




| Step | What happens | Code |
|---|---|---|
| 2 | Connects to the tenant's MCP servers and fetches tool *definitions* (names, descriptions, input schemas). Nothing is executed yet. | engine.py:40-41 |
| 3 | Builds the system prompt from the profile. Adds tool rules only if any tool exists. | engine.py:42 |
| 5 | Streams the LLM response. Text chunks are yielded to the user immediately; tool requests are collected. | engine.py:48-53 |
| 6-8 | If the LLM asked for tools: record the request, run each tool, append the results, then call the LLM again. | engine.py:57-80 |

# 3. The tool loop (sequence diagram)

The key idea: **the engine never decides to call a tool, the LLM does.** The engine only offers the tool list, executes what the LLM asks for, and returns the results so the LLM can write the final answer.




## Why a loop?

- One answer may need several tools (search availability, then book). Each round is one LLM call plus its tool executions.
- The loop runs at most **max_tool_rounds + 1 = 5** LLM calls so a confused model cannot loop forever.
- Messages grow each round: assistant *tool_use* blocks, then a user message of *tool_result* blocks. The LLM sees the full chain every time.
- If a tool fails, the result is sent with `is_error: true` and the LLM can apologise or retry.

## Tool routing

For each requested tool, the engine checks `mcp_tools.owns(name)`. If the tenant's MCP server owns it, it is executed remotely. Otherwise it goes to the built-in `ToolRegistry`.

# 4. Anatomy of the system prompt

The prompt is built **only from the domain profile**; there is no domain wording hard-coded in prompt.py. That is what makes the engine reusable across domains.

| | |
|---|---|
| **Identity** | You are the {persona} for {organization}... |
| **Style + date** | voice_style tone, today's date in tenant timezone |
| **Spoken-reply rules** | short, no markdown / lists / emojis |
| **KNOWLEDGE BASE** | FAQs (Q/A) and info key:value pairs. Empty for a new business |
| **RULES** | answer ONLY from KB or tools; else reply exact refusal line;<br>never invent prices or bookings; hand off to a human on request |
| **Tool rules (if tools exist)** | use tools for live data, confirm before changes,<br>exact date formats, never read out ids or JSON |
| **Collected entities** | details the caller already gave, so it never re-asks |

## The rule that matters most

```
- Answer organization-specific questions ONLY from the knowledge base above or from tool results.
- If the answer is not available, reply exactly: "I'm sorry, I don't have that information."
- Never invent availability, prices, bookings or policies.
```

This is a **grounding** rule: it stops the model from guessing, which is essential when a wrong price or booking has real consequences. The cost is that an assistant with no data refuses everything.

# 5. Why a new admin gets "I don't have that information"

This is what your screenshot showed. The greeting works because it comes from the profile, but a request like "I want to book" needs facts or tools, and there are neither.

**New admin, nothing connected**

Tools: none

Knowledge base: empty

Prompt rule: answer only from KB/tools

LLM has nothing to use

Reply: "I'm sorry, I don't have that information."

**MCP server connected**

Tools: search, book, cancel...

Knowledge base: optional FAQs

LLM requests a tool call

Engine executes it on the MCP server

LLM answers from live results

Reply: real availability / booking

## How to fix it

- **Connect an MCP server** on the MCP integration page. This unlocks live availability and booking.
- **Add knowledge-base entries** (FAQs and info) for static answers such as opening hours.
- **Ask in-domain questions.** A hotel business should be asked about rooms, not gyms.
- **Product improvement:** show a banner when a tenant has no tools and no knowledge base, so a first-time admin understands why the assistant refuses.

# 6. Concepts to learn, in order

Learn these roughly top to bottom. Each builds on the one before. The last column is where to see it in this project.

| # | Concept | What to understand | See it here |
|---|---|---|---|
| 1 | **HTTP APIs and REST** | Requests, responses, status codes, JSON, authentication cookies. | api/routes.py |
| 2 | **Python async / await** | Coroutines, `async for`, async generators (`yield` inside async). The engine streams this way. | engine.py |
| 3 | **Streaming (SSE / chunked)** | Sending text to the browser piece by piece so replies feel instant. | llm.stream() |
| 4 | **LLM basics** | Tokens, context window, system vs user vs assistant roles, temperature, the messages array. | ai/base.py |
| 5 | **Prompt engineering and grounding** | Writing rules that constrain the model; refusing when facts are missing; why exact-wording refusals help. | prompt.py |
| 6 | **Tool use / function calling** | The LLM returns a structured tool request; your code runs it and returns the result; JSON-schema tool definitions. | engine.py:45-80 |
| 7 | **MCP (Model Context Protocol)** | A standard way for an app to expose tools to any LLM client: servers, tool listing, tool calling, transports (streamable HTTP). | mcp/tools.py, GymBooking/mcp-server |
| 8 | **Multi-tenancy** | One deployment, many businesses. Every query and tool list is scoped by tenant_id from the session, never from client input. | for_tenant(), repositories |
| 9 | **Data modelling and migrations** | SQLAlchemy models, Alembic migrations, seed data, foreign keys. | models/entities.py, alembic/ |
| 10 | **Domain-driven configuration** | Behaviour as data (profiles) instead of code; one engine, many domains. | domain/catalog.py |
| 11 | **Voice pipeline (later)** | Speech-to-text, text-to-speech, telephony, latency budgets. Replies are short because they are spoken. | Start call button |
| 12 | **RAG (optional next step)** | Retrieving relevant documents into the prompt instead of pasting every FAQ; needed when the knowledge base grows. | future work |

# 7. Glossary and practice exercises

## Glossary

| Term | Meaning |
|---|---|
| **Tenant** | One business (for example Sunrise Grand Hotel) using the platform. |
| **Domain profile** | Stored config: identity, knowledge base and booking workflow for a tenant. |
| **System prompt** | Hidden instructions sent to the LLM before the conversation, built fresh every turn. |
| **Tool definition** | A name, description and JSON schema telling the LLM what a tool does and what input it needs. |
| **tool_use / tool_result** | The message blocks for the LLM's tool request and the engine's answer to it. |
| **MCP server** | A service exposing tools over the Model Context Protocol. The gym mock app has one. |
| **Grounding** | Forcing answers to come from supplied facts or tools, not the model's imagination. |
| **Streaming** | Delivering the reply in chunks as it is generated. |

## Practice exercises

- Add a print in `run_turn` showing `len(tool_defs)` and watch it change as you connect and disconnect an MCP server.
- Log the full `system` string for a new business and for one with FAQs. Compare them.
- Connect the Chennai gym MCP server, ask "yoga class tomorrow in Adyar", and trace every LLM call and tool call.
- Lower `max_tool_rounds` to 0 and observe what breaks. Why does the answer degrade?
- Write one FAQ in the knowledge base and confirm the assistant answers it with no MCP server connected.

Note: this guide describes engine.py and prompt.py as they exist in the repository today. File paths and line numbers may drift as the code changes.